PROOF OF CONCEPT

MATM MySQL → Unity Catalog Ingestion Framework

Project: `TDPTSXX-MATM-MySQL-ETL`
Audience: engineering / data platform walkthrough
Purpose: config-driven ingestion of Manhattan Active (MATM) MySQL data into Databricks Delta + Unity Catalog

1. The idea (why this exists)

MATM data lives in a MySQL replica (microservice databases behind Manhattan Active). We need that data in the lakehouse on a schedule — not as one-off notebooks, but as a repeatable ingestion product.

What we are implementing

A **MySQL → Delta / Unity Catalog** ingestion framework where:

1. Connection details and credentials live in one shared stub.
2. Each table/entity is defined in JSON (SQL, keys, target paths).
3. One runner drives every entity through the same three stages: **Extract → Transform → Completion**.
4. Jobs support **daily full** and **hourly/transactional incremental** loads.
5. Writes land in Unity Catalog and ADLS Delta (primary + secondary region).

Goals

- Pull MATM MySQL databases into Delta / UC reliably.
- Keep business logic in **configs**, not hardcoded notebooks.
- Reuse the same notebooks for every new table.
- Track incremental watermarks via loadtracker.
- Deploy as a Databricks Asset Bundle (dev / tst / stg / prd targets).

2. Source system

The source is the MATM MySQL replica hosted on **GCP Cloud SQL**. Its structure was captured from the generated *MySQL Database Documentation* report.

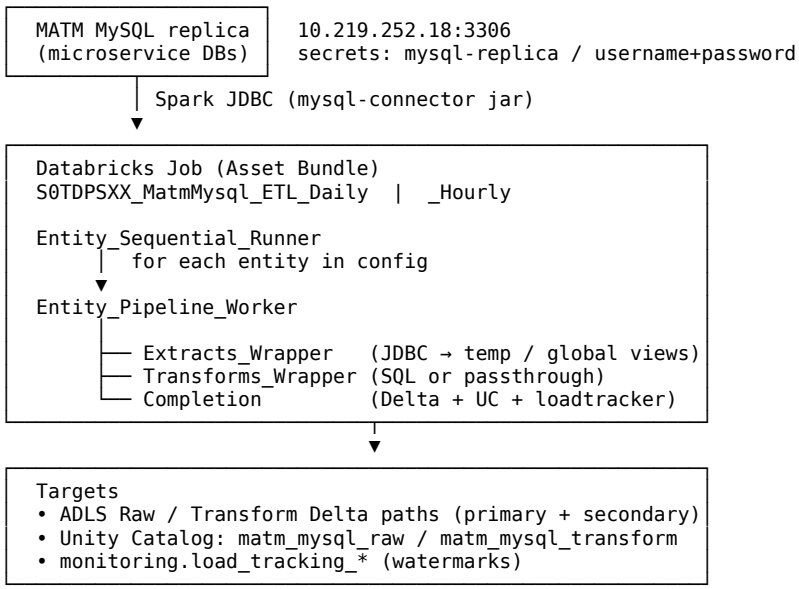
Item	Detail
Host	`10.219.252.18` (GCP Cloud SQL)
Databases	`default_carrier_shipment`, `default_shipment`, `default_vendor_shipment`

Database	Tables	Notes
`default_carrier_shipment`	188	Documented in the report
`default_shipment`	TBC	Count to be confirmed
`default_vendor_shipment`	TBC	Count to be confirmed

Example table (from the report): `BATCH_JOB_EXECUTION` in `default_carrier_shipment` is a BASE TABLE with an estimated row count of 0.

Each of these databases is referenced per entity through `jdbc.database_by_env`, while the shared host and credentials stay in the connection stub (see section 5).

3. High-level architecture



Config-driven control plane

Config	Role
`mysql_source_stub.json`	Shared host, port, driver, secret scope
`mysql_extract_daily.json`	Entity list + SQL for full daily loads
`mysql_extract_hourly.json`	Entity list + incremental flags for transactional loads

Add a new MATM table ≈ add a JSON entity block. No new notebook required for a standard extract.

4. End-to-end runtime flow

4.1 Job start

1. Job cluster starts with env vars from the bundle target:
   - `Databricks_Env` (`dev` / `tst` / `stg` / `prd`)
   - `Daily_Source_Config_Path` / `Transactional_Source_Config_Path`
   - `Target_Config_Path` (MySQL connection stub)

- Loadtracker catalog / schema / table

2. MySQL JDBC jar is attached via `MySql_Jdbc_Jar_Path`.
3. Entry notebook: `Entity_Sequential_Runner`.

4.2 Sequential runner

1. Loads source JSON (`entities` map).
2. Resolves which entities to run (`entity_name` widget empty = all; otherwise that entity + dependencies).
3. Ensures loadtracker table exists.
4. For each entity, calls `Entity_Pipeline_Worker` with the same paths + that entity name.
5. Aggregates successes / failures; fails the job if any entity failed.

4.3 Per-entity pipeline worker

Runs three notebooks in order:

```
Extracts_Wrapper  →  Transforms_Wrapper  →  Completion
```

4.4 Extract

1. Read entity `extract.query_by_env` for current `Databricks_Env`.
2. Build JDBC options from entity `jdbc` + `mysql_source_stub` (`connection_source: target_config`).
3. Optional incremental: if `incremental.enabled` and `load_mode=transactional`, append watermark predicate using last successful load from loadtracker (or fallback MAX on UC table).
4. `spark.read.format("jdbc")` → create temp + global temp views (`mysql_extract_*`).

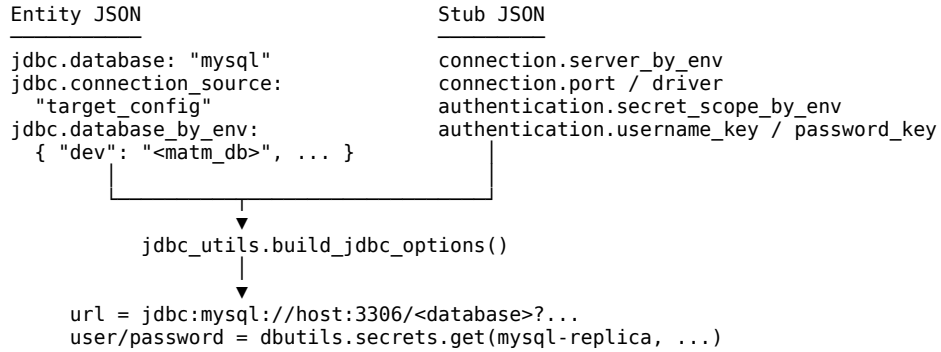
4.5 Transform

1. Read extract global temp view.
2. If `transform.query_by_env` is set → run that Spark SQL.
3. Else → **passthrough** (same columns/types as extract).
4. Publish `mysql_transform_*` views.

4.6 Completion

1. Resolve Delta / UC paths from entity `target.*_by_env`.
2. Dedupe on configured keys if needed.
3. Write Raw + Transform layers (primary + secondary region paths/catalogs).
4. Update loadtracker when incremental watermarks apply.
5. Hourly path can bootstrap a missing transform table with a full extract first.

5. Connection model (how MySQL auth works)



Why stub + entity split?

- One MATM host / credential set for all microservice DBs.
- Each entity overrides **which MySQL database/schema** via `jdbc.database_by_env` (e.g. one entity per MATM service DB).

Prerequisites on the workspace

- Network path from job cluster to `10.219.252.18:3306`
- Secret scope `mysql-replica` with `username` and `password`
- MySQL connector JAR on the UC volume path referenced by `MySql_Jdbc_Jar_Path`

6. Entity config shape (what we implement per table)

Each key under `entities` is one ingestible object. Important fields:

```
{
  "entity_name": "InformationSchemaTables",
  "jdbc": {
    "database": "mysql",
    "connection_source": "target_config",
    "database_by_env": { "dev": "information_schema", "...": "..." },
    "driver": "com.mysql.cj.jdbc.Driver",
    "options": { "fetchsize": "10000" }
  },
  "schema": {
    "source_table": "information_schema.TABLES",
    "columns": ["TABLE_SCHEMA", "TABLE_NAME", "..."]
  },
  "extract": {
    "enabled": true,
    "query_by_env": { "dev": "SELECT ... FROM ...", "tst": "...", "stg": "...", "prd": "..." },
    "temp_view_name": "mysql_extract_...",
    "global_temp_view_name": "mysql_extract_..."
  },
  "transform": {
    "enabled": true,
    "temp_view_name": "mysql_transform_...",
    "global_temp_view_name": "mysql_transform_..."
  },
  "completion": { "enabled": true },
  "incremental": {
    "enabled": false,
    "watermark_column": "UPDATE_TIME",
    "uc_watermark_column": "UPDATE_TIME",
    "loadtracker_entity_name": "InformationSchemaTables"
  },
  "target": {
    "write_enabled": true,
    "primary_key_columns": ["TABLE_SCHEMA", "TABLE_NAME"],
    "dedupe_key_columns": ["TABLE_SCHEMA", "TABLE_NAME"],
    "dedupe_order_columns": ["UPDATE_TIME", "CREATE_TIME"],
    "raw_merge_on_primary_key": false,
    "target_entity": "TDPS_MatmInformationSchemaTables",
    "target_uc_schema": "matm_mysql_raw",
    "target_file_path": "/Publix/SupplyChain/MatmMysql/Raw/",
    "target_file_format": "Delta",
    "target_datalake_by_env": { "dev": "abfss://...", "...": "..." },
    "secondary_datalake_by_env": { "...": "..." },
    "target_unity_catalog_by_env": { "dev": "ent_dtlk_dev", "...": "..." },
    "secondary_unity_catalog_by_env": { "dev": "entc_dtlk_dev", "...": "..." }
  }
}
```

Daily vs hourly

Mode	Job	Typical incremental	Write behavior
`daily`	Daily job	Usually off (full extract)	Full / overwrite style per delta utils
`transactional`	Hourly job	On when `incremental.enabled`	Merge on PK when configured

7. What ships today (sample implementation)

Starter entities prove the path end-to-end:

Entity	Source	Target entity name	UC schema
`InformationSchemaTables`	`information_schema.TABLES` (excl. system schemas)	`TDPS_MatmInformationSchemaTables`	`matm_mysql_raw`
`InformationSchemaColumns`	`information_schema.COLUMNS` (daily sample)	`TDPS_MatmInformationSchemaColumns`	`matm_mysql_raw`

Lake root: `/Publix/SupplyChain/MatmMysql/Raw/` (Transform derived by replacing `Raw` → `Transform`).

These are **scaffolding / metadata** entities. Real MATM business tables are added the same way once owners pick databases and PKs.

8. Jobs & schedules

Job name	Load mode	Config	Schedule (default, paused)
`S0TDPSXX_MatmMysql_ETL_Daily`	`daily`	`mysql_extract_daily.json`	`9 0 22 ?` America/New_York
`S0TDPSXX_MatmMysql_ETL_Hourly`	`transactional`	`mysql_extract_hourly.json`	`12 15 * ?` America/New_York

Job parameters

- `entity_name` — empty = all entities; set to run one
- `continue_on_error` — default `true` on the job definition

Jobs are created **paused**; enable after jar + secrets + network are validated.

9. How to add a real MATM business table

1. Identify MySQL database name (MATM microservice) and table.
2. Copy an entity block in `mysql_extract_daily.json` (and hourly if needed).
3. Set:
   - `jdbc.database_by_env` → that microservice DB
   - `extract.query_by_env` → explicit column SELECT (avoid `SELECT *` for schema validation)
   - `schema.columns` matching the SELECT list
   - `target.primary_key_columns` / dedupe columns
   - `target.target_entity` naming convention (`TDPS_Matm...`)
   - Incremental watermark column if hourly
4. Deploy bundle; run job with `entity_name=<YourEntity>`.
5. Validate UC table + Delta path + loadtracker row.

No new Python notebooks required for a standard ingest.

10. Deploy & validate (operator checklist)

```
cd TDPTSXX-MATM-MySQL-ETL
databricks bundle validate -t DEV_East
databricks bundle deploy -t DEV_East
```

Then:

1. Confirm MySQL jar exists at the volume path in `targets/dev.yml`.

2. Confirm secret scope `mysql-replica` on the workspace.
3. Run Daily job with `entity_name=InformationSchemaTables`.
4. Check UC: `ent_dtlk_dev.matm_mysql_raw.TDPS_MatmInformationSchemaTables` (names per env).
5. Check loadtracker if using hourly/incremental.
6. Unpause schedules when ready.

Azure DevOps pipeline: Dev East/Central → Tst East/Central (`azure-pipelines/`).